QUESTION1a) In this situation, it talks about finding the time for a first-order stable system (single link robot manipulator) to reach 98% of its steady-state value, and about how the position, speed, and acceleration change during this time.

SOLUTION1a) The main parameter that shows the response speed of a first-order system is the time constant. This is called the **time constant** (τ).The system reaches 98.2% of its final value in 4 tau. It is usually accepted as 98%. Because the question says "98% of its steady state value," the end time of the simulation is chosen as 4τ. This is also called the settling time.
The term e^{-at} in the given equation shows that the pole of the system is $-a$. I calculated the time constant (τ) as the inverse of the absolute value of the pole.

$$\tau = \frac{1}{|a|} \quad (1a.1)$$

For a=2

$$\tau = \frac{1}{|2|} = 0.5 \ second \ (1a.2)$$

For the target 98% settling time:

$$T_{final} = 4 \times \tau = 4 \times 0.5 = 2 \ second \ (1a.3)$$

This result shows that the simulation must run from t = 0 to t = 2.
The steady state gain is the response of the system when $t \to \infty$. It should be calculated by using the final value theorem, by taking the limit of the transfer function as $s \to 0$.
The general transfer function of a first-order system:

$$G(s) = \frac{b}{s+a} \quad (1a.4)$$

$$SSG = \lim_{s \to 0} G(s) = \frac{b}{0+a} = \frac{b}{a} \quad (1a.5)$$

Because the steady state gain of the system is given as 2:
By putting 4 instead of b, the full mathematical model of the system is obtained.

$$\frac{b}{a} = 2 \rightarrow \frac{b}{2} = 2 \rightarrow b = 4 \ (1a.6)$$

For the simulation, we need the position, speed and acceleration equations of the system. In the given solution, $x(t)$show the position. The speed is the first derivative of the position and acceleration is the second derivative of the position.
Position(x(t)):

$$x(t) = x(0)e^{-at} + \frac{b}{a}\left(1 - e^{-at}\right)u(t) \quad (1a.7)$$

The values $x(0) = 0.2, a = 2, b = 4, u(t) = 1$should be used in the equation:

$$x(t) = 0.2e^{-2t} + \frac{4}{2}\left(1 - e^{-2t}\right)$$

$$x(t) = 0.2e^{-2t} + 2\left(1 - e^{-2t}\right) \quad (1a.8)$$

This $x(t)$ equation gives the time-dependent angular position of the robot arm.
Speed(V(t)):
It is derivative of the position with respect to time.

$$V = \frac{dx}{dt} \quad (1.9)$$

$$V(t) = \frac{d}{dt}\left[0.2e^{-2t} + 2 - 2e^{-2t}\right] (1a.10)$$

$$V(t) = 0.2(-2)e^{-2t} + 0 - 2(-2)e^{-2t} \ (1a.11)$$

$$V(t) = -0.4e^{-2t} + 4e^{-2t} \quad (1a.12)$$

$$V(t) = 3.6e^{-2t} \quad (1a.13)$$

This equation gives the angular speed of the robot arm. At the start, when t equals 0 the speed takes its highest value which is 3.6 However,as time goes to infinity it approaches zero.
Acceleration(a(t)):
Acceleration is the derivative of speed with respect to time.

$$a = \frac{dv}{dt} = \frac{d^2x}{dt^2} \quad (1a.14)$$

$$a(t) = \frac{d}{dt}\left[3.6e^{2t}\right] \quad (1a.15)$$

$$a(t) = 3.6(-2)e^{-2t} \ (1a.16)$$

$$a(t) = -7.2e^{-2t} \quad (1a.17)$$

The reason I derived from this equation is that it gives the acceleration of the robot arm. If the acceleration is negative, the speed of the system decreases and it slows down over time.x0 = 0.2;

```
a = 2;
SSG = 2;
b = SSG * a;
tau = 1/a;
tfinal = 4*tau;
dt = 0.01;
t = 0:dt:tfinal;
x0 = 0.1;
x = x0*exp(-a*t) + (b/a)*(1 - exp(-a*t));
v = (b - a*x0)*exp(-a*t);
acc = -a*(b - a*x0)*exp(-a*t);
figure;
subplot(3,1,1);
plot(t, x,'LineWidth',2);
title('Position');
ylabel('Rad');
grid on;
subplot(3,1,2);
plot(t, v,'LineWidth',2,'Color','r');
title('Velocity');
ylabel('Rad/s');
grid on;
subplot(3,1,3);
plot(t, acc,'LineWidth',2,'Color','b');
title('Acceleration');
ylabel('Rad/s^2');
xlabel('Time (s)');
grid on;
```

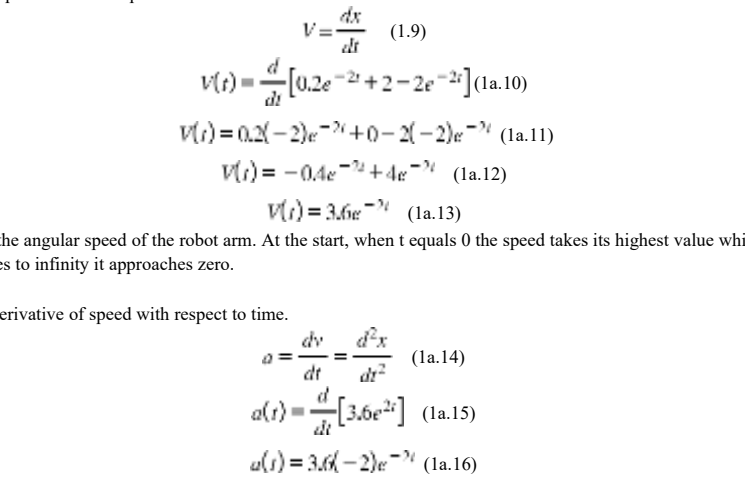


Figure 1.1 POSITION , VELOCITY AND ACCELERATION OF THE STABLE

QUESTION1b) This part wants the dynamic state matrix that collects the kinematic variables derived from the simulation parameters set by the user.

SOLUTION1b) In this case, I made the continuous-time solution from part (a) discrete using the sampling time (Δt) that I set. To keep the data sizes the same, I padded the velocity and acceleration vectors, found by numerical differentiation, to match the length of the position vector. Finally, I joined these vectors row by row to make a single 3×N system matrix, where N changes according to the simulation time I set.
In this question, I used the "Data Acquisition & Structure Formulation" process.
Since the simulation is digital, the continuous-time derivatives were calculated using the finite difference method and converted into a discrete-time array.

- **Position vector:** It creates an array with the data obtained from the analytical solution according to the time steps.

$$x[k] = x(t_k) \quad \text{for } k = 0, 1, \ldots N \ (1b.1)$$

- **Velocity Calculation (Numerical Derivative):** The MATLAB "diff" function was used, which corresponds mathematically to the forward difference method.(Δt is the time step dtentered by the user.)

$$v[k] \approx \frac{x[k+1] - x[k]}{\Delta t} \quad (1b.2)$$

- **Acceleration Calculation:** It should be obtained by taking the derivative of the velocity data.

$$a[k] \approx \frac{v[k+1] - v[k]}{\Delta t} \quad (1b.3)$$

- **Matrix Structure (Matrix Concatenation):**The obtained vectors (of size $1 \times N$) are stacked vertically (row-wise concatenation) according to linear algebra notation.

$$\mathbf{M}_{system} = \begin{bmatrix} x[0] & x[1] & \ldots & x[N] \\ v[0] & v[1] & \ldots & v[N] \\ a[0] & a[1] & \ldots & a[N] \end{bmatrix}_{3 \times N} \quad (1b.4)$$

1) **Dynamic Memory Management:** The vector length (N) is determined at runtime using the T_{final} value received from the user. (T_{initai} is zero.)

$$N = \frac{T_{final}}{\Delta t} + 1 \quad (1b.5)$$

2) **Size Preservation (Padding):** The numerical derivative (diff) reduces the data set length by 1 ($N - 1$). To keep the matrix rectangular and avoid size errors, the last calculated value is added again at the end of the derivative vectors.

$$v_{final} = [v_{calc}, v_{calc}(end)] \ (1b.6)$$

3) **Matrix Construction:** The calculated vectors, with equal lengths are combined into a single data structure using the [;] operator.

```
x0 = 0.2;
a = 2;
ss_gain = 2;
b = ss_gain * a;
dt = input(' Enter step size (dt) : ');
t_upper = input(' Enter upper time limit: ');
t_vec = 0:dt:t_upper;
x = x0 * exp(-a * t_vec) + (b/a) * (1 - exp(-a * t_vec));
dx = diff(x);
v_calc = dx / dt;
v = [v_calc, v_calc(end)];
dv = diff(v);
a_calc = dv / dt;
acc = [a_calc, a_calc(end)];
System_Matrix = [x; v; acc];
[r, c] = size(System_Matrix);
fprintf(' Matrix size: %d x %d\n ', r, c);
```

```
Command Window
Enter step size (dt) : 0.02
Enter upper time limit: 2
Matrix size: 3 x 101
```

QUESTION1c) This part focuses on taking the final data from the dynamic system matrix to study how the system states move toward steady-state.

SOLUTION1c) In this case, because the number of columns in the system matrix changes with the user-defined simulation time, we cannot use fixed indexing. Therefore, we use dynamic indexing with the end operator to safely extract the last five columns of the main $3 \times N$ dynamic system matrix. This produces a smaller 3×5 matrix that shows the system behavior at the end, as $t \to T_{\text{final}}$. These final values show that the position is almost at steady-state ($x_{ss} \approx 2$) and the velocity and acceleration are nearly zero, meaning the motion has settled. The last five samples are chosen because the question asks for the terminal five values, which represent the steady-state part where dynamic changes are very small.

```
a_pole = 2;
ss_gain = 2;
x_init = 0.2;
dt = 0.02;
b_parametre = ss_gain * a_pole;
t_user = input(' Enter upper time limit: ');
t_vec = 0:dt:t_user;
x_val = x_init * exp(-a_pole * t_vec) + (b_parametre / a_pole) * (1 - exp (-a_pole * t_vec));
dx = diff(x_val);
v_val = dx / dt;
v_val = [v_val, v_val(end)];
dv = diff(v_val);
acc_val = dv / dt;
acc_val = [acc_val, acc_val(end)];
System_Matrix = [x_val; v_val; acc_val];
Terminal_Matrix = System_Matrix(:, end-4:end);
disp(Terminal_Matrix);
```

```
Command Window
Enter upper time limit: 2
    1.9613    1.9628    1.9643    1.9657    1.9670
    0.0758    0.0729    0.0700    0.0673    0.0673
   -0.1487   -0.1429   -0.1373         0         0
```

QUESTION2a) It talks about modeling the change of the unstable pole of the system (a = -2) over time, and studying how this change affects the dynamic response by rewriting the differential equation and simulating it.

SOLUTION2a) In this situation, it is obvious that the stability parameter *a* is no longer constant. I thought that it would change over time due to physical degradation, and therefore the system would become Linear Time-Varying (LTV). Consequently, I considered that it can no longer be regarded as Linear Time-Invariant (LTI) for the remaining period.This change indicates the presence of internal parametric uncertainty. To model this correctly, I updated the differential equation at every simulation step using the value of *a(t)*, which decreases linearly from its initial value.
In this case, the coefficient of the differential equation must be updated each time step.
The initial value is $a_{\text{start}} = -2$ In 5 seconds, this parameter undergoes 10% erosion. Erosion also means that it gets closer to zero.

- At the t = 0, a(0) = −2
- At the t = 5, a(5) = −2(1 − 0.10) = −1.8

It is written as a linear function:

$$a(t) = a_{nom} \times \left(1 - \text{ErosionRate} \times \frac{t}{T_{erosion}}\right) \quad (2a.1)$$

$$a(t) = -2 + \left(\frac{-1.8 - (-2)}{5}\right) \cdot t = -2 + \left(\frac{0.2}{5}\right) \cdot t = -2 + 0.04t \quad (2a.2)$$

Re-expressed ODE form:

$$\frac{dx(t)}{dt} + (-2 + 0.04t) \cdot t = 1u(t) \quad (2a.3)$$

```
dt = 0.01;
t_final = 5;
t_vec = 0:dt:t_final;
a_nom = -2;
b = 1;
u = 1;
x0 = 0.1;
ero_pct = 10;
ero_time = 5;
x = zeros(1,length(t_vec));
x(1) = x0;
for k = 1:length(t_vec)-1
t_now = t_vec(k);
ero = (ero_pct/100) * (t_now/ero_time);
a_t = a_nom * (1 - ero);
dx = -a_t*x(k) + b*u;
x(k+1) = x(k) + dx*dt;
end
figure;
plot(t_vec,x,'r','LineWidth',2);
grid on;
xlabel('Time (s)');
ylabel('Position');
title(['Unstable System Response with ' num2str(ero_pct) '% Parameter Erosion']);
fprintf('Initial Pole a(0): %.2f\n', a_nom);
fprintf('Final Pole a(%.1f): %.2f\n', ero_time, a_t);
```

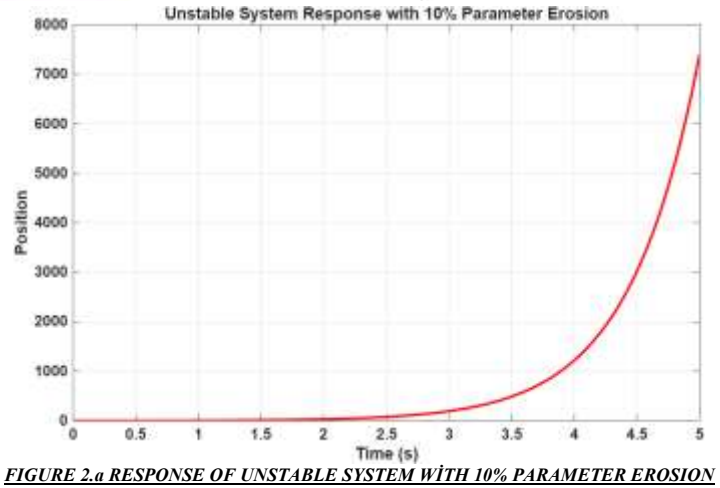


FIGURE 2.a RESPONSE OF UNSTABLE SYSTEM WİTH 10% PARAMETER EROSION

```
Command Window
Initial Pole a(0): -2.00
Final Pole a(5.0): -1.80
```

QUESTION2b) This involves mathematically expressing an external disturbance that acts on the system for an infinitely short time and simulating the 10% sudden jump it creates in the instantaneous state.

SOLUTION2b) Under these conditions, disturbances acting on the system are classified as External Non-Parametric Disturbance. Effects that happen in an infinitely short time and cause a sudden energy transfer or position change in the system are modeled by the Dirac delta function or impulse function (δ(t)). When the impulse input is added to the differential equation, taking the integral causes an instantaneous step (jump) change in the state variable.
In this case, the disturbance signal is not a continuous function it is an impulse that exists at a specific time $t_{disturbance}$ and is zero at other times. Therefore the mathematical model of the system, that is the ODE, is rewritten to include this external effect as an input term.

The basic dynamic equation of the system (unstable mode):

$$\frac{dx(t)}{dt} + ax(t) = bu(t), \quad a = -2 \ and \ b = 1$$

The disturbance $d(t)$ acting on the system at time $t_{disturbance}$ is added to the system equation as an additive term.

$$dt = A \cdot \delta(t - t_{disturbance})$$

represents the unit function, and Arepresents the strength of the impulse, that is, the jump amount. Since the question says "10% instant state jump," the effect at the jump depends on the current value of $x(t)$. For example at t = 0 it is equal to $(0.10) \cdot x(t)$.
Re-expressed ODE:

$$\frac{dx(t)}{dt} - 2x(t) = 1 \cdot u(t) + d(t) \quad , \quad d(t) \ is \ external \ disturbance \quad (2b.1)$$

For the simulation:

$$\frac{dx(t)}{dt} = 2x(t) + u(t) + (The \ value \ added \ at \ the \ jump \ instant) \quad (2b.2)$$

```
dt = 0.01;
tfinal = 5;
t = 0:dt:tfinal;
x = zeros(1, length(t));
x(1) = 0.1;
u = 1;
a = -2;
b = 1;
tdist = 2.5;
jumpratio = 0.10;
for k = 1:length(t)-1

    dx = -a * x(k) + b * u;
    x(k+1) = x(k) + dx * dt;
    if abs(t(k) - tdist) < dt/2

        jumpvalue = x(k+1) * jumpratio;
        x(k+1) = x(k+1) + jumpvalue;

        fprintf('Disturbance Applied at t = %.2f s. State Jumped by %.4f\n', t(k), jumpvalue);
    end
end
figure;
plot( t, x, ' b ', ' LineWidth ', 2 );
title([' Unstable System Response with External Disturbance t=' num2str(tdist) 's']);
xlabel('Time (s)');
ylabel('Position (State x)');
grid on;

hold on;
x_at_dist = interp1(t, x, tdist);
plot(tdist, x_at_dist, 'ro',' MarkerSize ', 10,' LineWidth ',2);
legend(' System Response ', ' Instant 10% Jump ');
```

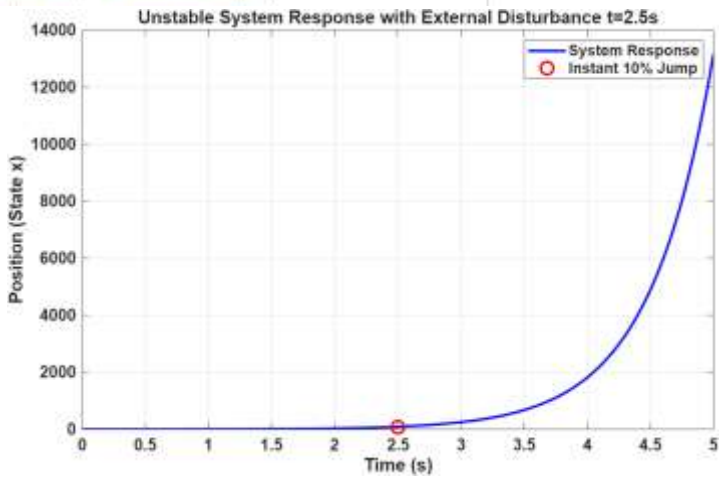


FIGURE 2.b UNTSTABLE SYSTEM RESPONSE WİTH EXTERNAL DİSTURBANCE T=2.5s

```
Command Window
Disturbance Applied at t = 2.50 s. State Jumped by 8.5956
```

QUESTION2c) In this part, we are asked to show the full closed-loop block diagram of the system.. A schematic showing how the controller,the system and the disturbances interact with each other is required.

SOLUTION2c) In this situation, the closed-loop system with $G(s) = \frac{1}{s-2}$ has a pole $p = 2$ in the Right-Hand Plane (RHP). This means the system is unstable by nature, and with any disturbance, the state variable will go to infinity. That is why I reached this conclusion. In such cases, negative feedback is used to keep the system in a safe and stable working area.
A standard single-input single-output (SISO) closed-loop system consists of four main components and two summing junction:

- Reference input(R(s)): The signal that the system is required to follow.
- Summing junction: The point where the error signal is generated. $\rightarrow E(s) = R(s) - Y(s)$
- Error signal(E(s)): The difference between the input and the output; it is the input to the controller.
- Controller(C(s)): The element that takes the error signal and generates the control signal (U(s)) to be applied to the system (equal to the gain Kp). $\rightarrow U(s) = C(s) \cdot E(s)$
- System (plant) (G(s)): The unstable system that receives the control signal and produces the physical response Y(s). $\rightarrow Y(s) = G(s) \cdot U(s)$
- Output Y(s): The actual measured response of the system. It is sent back to the summing junction as feedback.
- Feedback: Unit feedback is used, H(s)=1.

The closed-loop transfer function $T(s)$ define the relationship between the system output $Y(s)$and the reference input $R(s)$.

$$\frac{Y(s)}{R(s)} = \frac{G(s) \cdot C(s)}{1 + G(s) \cdot C(s)} \quad (2c.1)$$

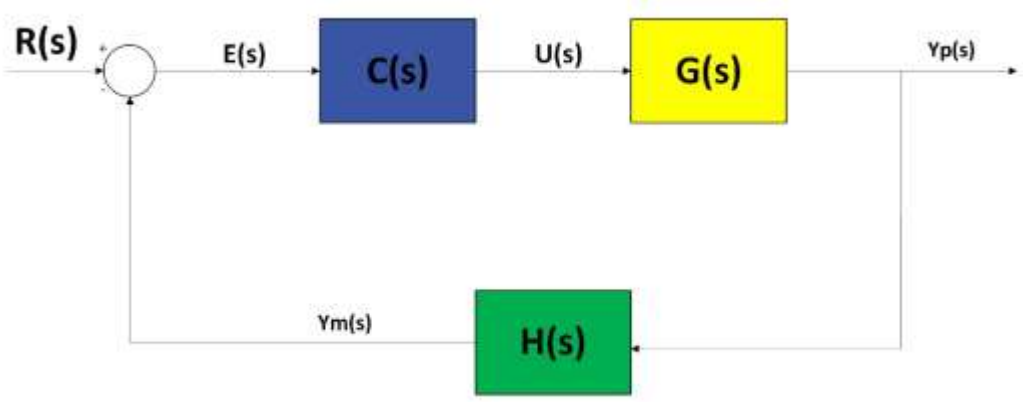


FIGURE 2.c OVERALL CLOSED LOOP DİAGRAM

The derivation of the equation is as follow:

1) $Y(s) = G(s) \cdot U(s)$
2) $U(s) = C(s) \cdot E(s)$
3) $E(s) = R(s) \cdot Y(s)$

Substituting the 2nd and 3rd equations into the 1st equation:

$$Y(s) = G(s) \cdot [C(s) \cdot E(s)] \quad (2c.2)$$

$$Y(s) = G(s) \cdot C(s) \cdot [R(s) \cdot Y(s)] \quad (2c.3)$$

$$Y(s) = G(s) \cdot C(s) \cdot R(s) - G(s) \cdot C(s) \cdot Y(s) \quad (2c.4)$$

The closed-loop transfer function $T(s)$ is obtained. This equation forms the basis of the characteristic equation used for stability analysis.

$$T(s) = \frac{Y(s)}{R(s)} = \frac{G(s) \cdot C(s)}{1 + G(s) \cdot C(s)} \quad (2c.5)$$

QUESTION2d) This section shows the design of a proportional controller using the nominal plant model. It also checks how the controller works when the system has parameter changes and external disturbances.

SOLUTION2d) In this situation, I used the nominal model during the design. However, in real life system parameters may change over time and external effects may disturb the system, just like in parts (a) and (b). For this part, I assumed that an uncertainty-free controller design was required. In other words, I designed the controller based solely on the ideal mathematical model without considering parameter changes or disturbances. In addition, I can say that a robust control system is the ability to remain stable and maintain good performance despite model uncertainties and external disturbances.
The design goal is to make the closed-loop system have a time constant (τ) of 2 seconds.
Determination of Nominal Open Loop Model Differential equation of the system.

$$\frac{dx(t)}{dt} + a_{nom} x(t) = b \cdot u(t)$$

Here, $a_{nom} = -2$(an unstable pole) and $b = 1$are given. When we take the Laplace transform, we get:

$$sX(s) - 2X(s) = U(s) \Rightarrow \frac{X(s)}{U(s)} = G(s) = \frac{1}{s-2} \quad (2d.1)$$

The open-loop pole is $s = +2$ (in the Right Half-Plane, so it is unstable)
Closed-Loop Transfer Function ($T(s)$) When a P-controller ($C(s) = K_p$) is added to the system and negative feedback is used:

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)} = \frac{K_p \cdot \frac{1}{s-2}}{1 + K_p \cdot \frac{1}{s-2}} \quad (2d.1)$$

$$T(s) = \frac{K_p}{s - 2 + K_p} \quad (2d.2)$$

The denominator here is the Characteristic Equation: $s + (K_p - 2) = 0$

Target Pole and Gain Calculation (Pole Placement) A first-order system has the standard characteristic equation $s + p_{target} = 0$ and The relation between the time constant (τ) and the pole (p) is:

$$\tau = \frac{1}{p_{hedef}} \rightarrow p_{hedef} = \frac{1}{\tau} \quad (2d.3)$$

Since the question ask for $\tau = 2$ seconds:

$$p_{hedef} = \frac{1}{2} = 0.5 \quad (2d.4)$$

Finally, we need to move the characteristic equation toward the target:

$$s + (K_p - 2) = s + 0.5 \ , \ K_p - 2 = 0.5 \ , \ K_p = 2.5 \quad (2d.5)$$

```
a0 = -2;
b = 1;
tau = 2;
ref = 1;
ero_pct = 10;
ero_time = 5;
dist_t = 4;
jump_p = 10;
dt = 0.01;
sim_time = 10;
t = 0:dt:sim_time;
pole = 1 / tau;
Kp = pole - a0;
x = zeros(1,length(t));
x(1) = 0;
for k = 1:length(t)-1
tk = t(k);
if tk <= ero_time
    erate = ero_pct/100;
    a = a0 * (1 - erate * (tk / ero_time));
else
    a = a0 * (1 - ero_pct/100);
end
e = ref - x(k);
u = Kp * e;
dx = -a*x(k) + b*u;
x(k+1) = x(k) + dx*dt;
if abs(tk - dist_t) < dt/2
    jmp = x(k+1)*(jump_p/100);
    if abs(jmp) < 0.1
        jmp = 0.2;
    end
    x(k+1) = x(k+1) + jmp;
end
end
figure;
plot(t,x,'b','LineWidth',2); hold on;
yline(ref,'--k','LineWidth',1.5);
xline(dist_t,'--r','LineWidth',1.5);
xline(ero_time,':g','LineWidth',1.5);
title([' Control Simulation (Kp = ' num2str(Kp) ')']);
xlabel('Time (s)');
ylabel('Output');
legend('System','Reference','Disturbance','Erosion End');
grid on;
fprintf('Kp = %.2f\n', Kp);
```

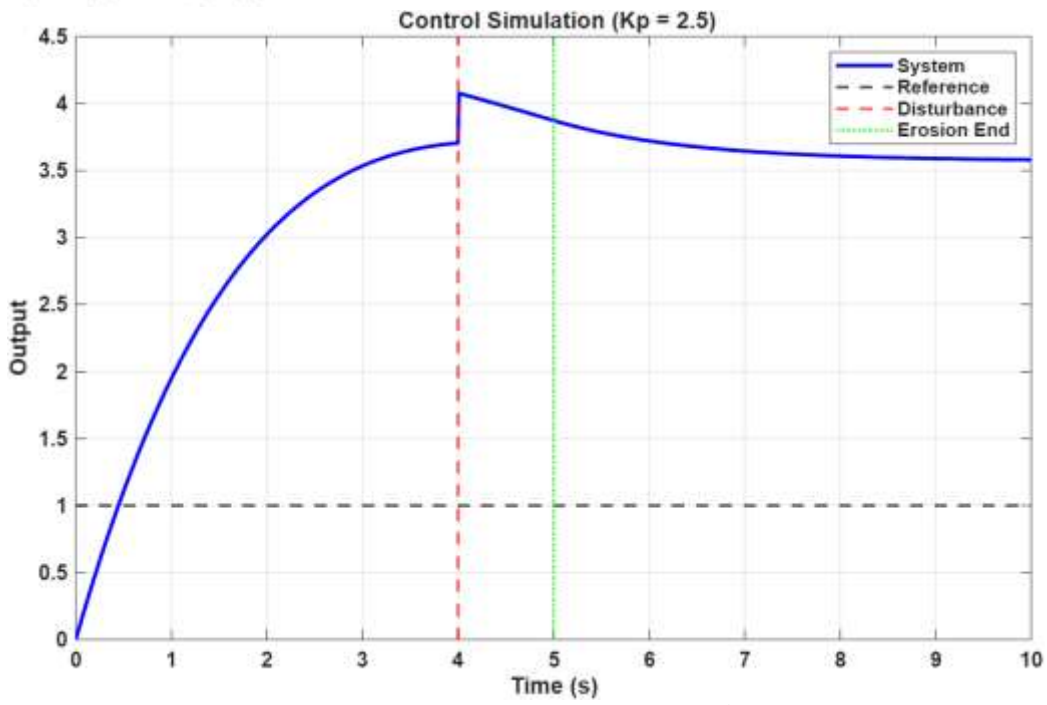


FIGURE 2.d CONTROL SIMULATION (K_p = 2.5)

```
Command Window
Calculated Kp: 2.50
```